

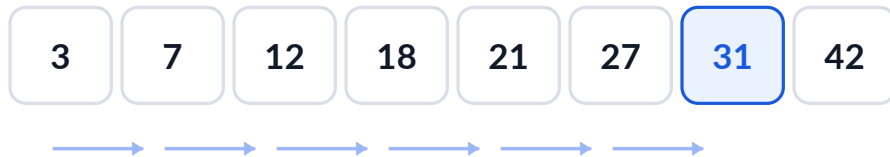
Finding a Value Efficiently

Linear search checks each item; binary search repeatedly halves the range.

LINEAR SEARCH

One item at a time

No ordering requirement



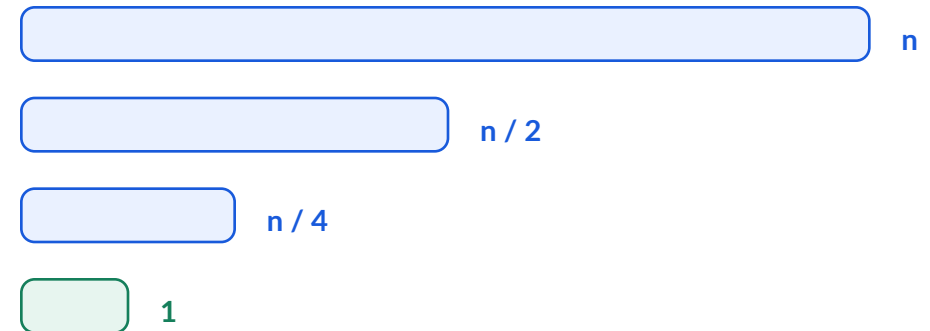
Worst case

$O(n)$ checks

BINARY SEARCH

Discard half each step

Requires sorted data



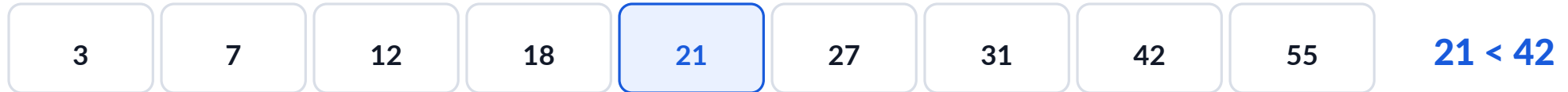
$O(\log n)$ checks

How Binary Search Works

Target: 42

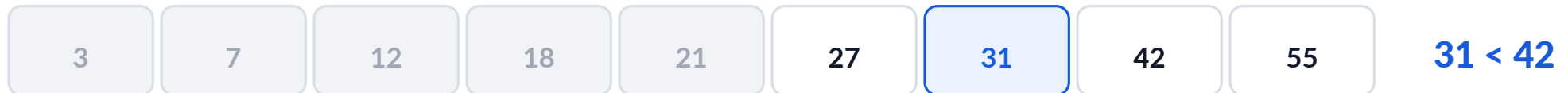
STEP 1

Inspect the midpoint



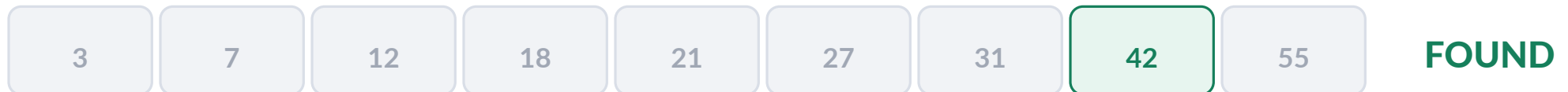
STEP 2

Discard the half that cannot contain the target



STEP 3

Repeat until the target is found

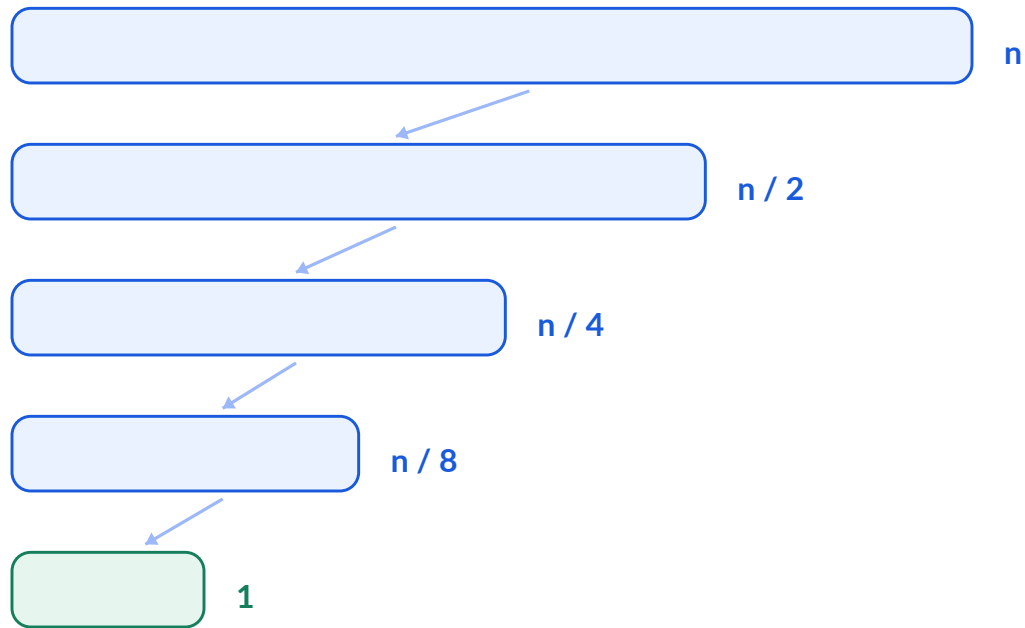


Compare -> discard half -> repeat

Why the Time Complexity Is $O(\log n)$

Each comparison removes half of the remaining candidates.

SEARCH SPACE



DERIVATION

$$n / 2^k = 1$$

$$2^k = n$$

$$k = \log_2(n)$$

Therefore

$$O(\log n)$$

Example: 1,024 candidates require about 10 halvings.

When Binary Search Works

Fast lookup depends on the right preconditions and implementation.

PRECONDITION

Sorted data

- The search range must be ordered.
- Without ordering, the half-discard rule is invalid.

COST TRADE-OFF

Sorting is not free

- Sorting unsorted data costs $O(n \log n)$.
- For one small search, linear search may be cheaper.

COMMON VARIANTS

Define the exact answer

- First occurrence
- Last occurrence
- Lower bound / upper bound

SORTED DATA + MIDPOINT COMPARISON + DISCARD HALF = $O(\log n)$

Standard binary search on a sorted array has worst-case time complexity $O(\log n)$.